

GatherGo

Version 1 — Product Requirements & Milestone Plan

February 2026 | 7 Milestones | 7 Weeks

Prepared by Zyntrix | Client: Go Gather | Version: v1

Project Overview

GatherGo is a group travel planning app that gives travellers a single shared space to organise trips and events, split expenses, store memories, connect with friends, and get AI-powered travel help. An Admin Web Panel provides Zyntrix and the client's team with full operational control.

Version 1 spans 7 milestones across 7 weeks — one milestone per week — covering the mobile app (React Native), backend API (Node.js on AWS), and admin web application (Next.js).

Seven Milestones at a Glance

#	Milestone	Scope Summary
M1	Auth & Profiles	Splash, Login, Signup, Google OAuth, Create Profile, Figma updates
M2	Home Screen & Trips	Home dashboard, Trip creation, Trip detail (Activities, Expenses, Docs, Members, Photos, Polls, Notes)
M3	Events	Event creation, Event detail (Docs, Members, Photos, Expenses, Polls, Notes) — no Activities
M4	Friends & Invite System	Connected friends, Invite friends, User profile with past trips & events gallery
M5	Gallery & AI Chatbot	Per-trip/event gallery, Personal gallery, Swee AI assistant (OpenAI GPT-4o)
M6	Revisions, Changes & Deployment	Client revisions, QA & polish, AWS backend deployment, app store submission
M7	Admin Web Panel	Next.js dashboard — Business Insights, Users, Trips/Events, Error/Health, Storage, Feedback

Technology Stack

Layer	Technology
Mobile Frontend	React Native — iOS and Android from single codebase. iOS 14+, Android 8+.
Admin Frontend	Next.js — Server-side rendered web application for admin panel.
Backend	Node.js + Express.js — RESTful API, modular route structure per feature module.
Database	AWS RDS (PostgreSQL) — Managed relational database with automated backups, multi-AZ.
File Storage	AWS S3 — All media, docs, and profile photos. Pre-signed URLs, CDN delivery via CloudFront. 50MB per file cap.
Authentication	Google OAuth 2.0 + JWT — Access token (15 min) + refresh token (30 days). Mobile: Keychain/Keystore.
Email	AWS SES — Transactional: invite emails, password reset, reminders. No marketing.
AI Assistant	OpenAI API (GPT-4o) — Trip/event context injected via system prompt. Streaming responses.
Push Alerts	Firebase Cloud Messaging — Cross-platform push for iOS and Android.
Maps / Places	Google Places API — Location search in Create Trip/Event modals.
CI/CD	GitHub Actions — Lint, test, build on every push to main. Auto-deploy on green build.
Hosting	AWS (EC2 / Elastic Beanstalk) — Node.js backend + Next.js admin under HTTPS with SSL via AWS ACM.

What is OUT of Scope for v1

- Community / Social Feed
- Group Chat between members (Swee AI only in scope)
- Multi-currency support (v1 defaults to INR)
- In-app payments or wallet integration
- Any feature not explicitly listed in the seven milestones above

M1 — Auth & Profiles

Week 1

Foundation: authentication flows, secure sessions, user profile creation, and Figma designs for all v1 screens.

Milestone Goals

- Mobile project bootstrapped: folder structure, navigation, theming, bottom navigation shell.
- Splash screen with auto-login check (JWT token validation).
- Login screen: email + password and Google OAuth 2.0.
- Signup screen: email, phone, country code picker, password, confirm password.
- Create Profile screen: name, DOB, gender, country, short bio, optional profile photo.
- Secure session management: JWT access (15 min) + refresh token (30 days) in Keychain/Keystore.
- FCM token captured at login and stored for push notifications.
- Node.js backend scaffold with Express, AWS RDS connection, GitHub Actions CI.
- Figma designs for all v1 screens delivered and approved by client before Week 2 begins.
- All screens fully responsive on iOS and Android.

Screen Specifications

Splash Screen

- App logo centred on teal background. Tagline: 'Gather & Go: Plan your next shared adventure!'
- Auto-check: validate stored JWT via GET /auth/me.
- If expired: silent refresh via POST /auth/refresh. If refresh fails: redirect to Login.
- Navigates to Home Dashboard in under 2 seconds if token is valid.

Login Screen

- Two tabs: Login and Create Account.
- Combined email-or-phone field with real-time format detection. Password: masked, show/hide toggle.
- Log In button disabled until both fields are non-empty.
- Forgot Password: sends reset email via AWS SES (POST /auth/forgot-password).
- Continue with Google: opens Google OAuth 2.0 consent flow.
- Create Account tab: email, country code dropdown (pre-filled +91 IN), 10-digit phone, password min 8 chars, confirm password match.
- Terms & Conditions link below the Continue button.

Create Profile Screen

- Circular avatar with tap-to-upload (camera or photo library).
- Full Name (required, min 2 chars), Date of Birth (DD/MM/YYYY), Gender dropdown, Country searchable dropdown.
- Short bio / tagline (optional, max 120 chars).

- Save & Continue: calls POST /users/profile, then navigates to Home Dashboard.

Backend API Endpoints — M1

Method & Route	Description	Key Fields
POST /auth/signup	Email/password registration	body: {email, phone, password, fcmToken} → {userId, accessToken, refreshToken}
POST /auth/login	Email/password login	body: {emailOrPhone, password, fcmToken} → {userId, accessToken, refreshToken}
POST /auth/google	Google OAuth token exchange	body: {googleIdToken, fcmToken} → {userId, accessToken, refreshToken, isNewUser}
POST /auth/refresh	Silent token refresh	body: {refreshToken} → {accessToken, refreshToken}
POST /auth/logout	Invalidate session	header: Bearer → {success: true}
POST /auth/forgot-password	Send password reset email	body: {emailOrPhone} → {success: true}
GET /auth/me	Get current user from token	header: Bearer → {userId, email, profile}
POST /users/profile	Save profile after signup	body: {fullName, dob, gender, country, bio} → {profile}
PUT /users/photo	Upload profile photo to S3	multipart/form-data → {photoUrl}
GET /users/:id	Get user public profile	header: auth → {profile}

Acceptance Criteria — M1

- User signs up with email/password and is redirected to Create Profile on first login.
- User logs in via Google OAuth without entering a password.
- JWT access token expires in 15 min; silent refresh works without user intervention.
- Logout clears tokens from device and redirects to Login.
- Create Profile saves all fields; user name displays on Home Dashboard.
- Figma designs for all v1 screens delivered and client-approved before Week 2.
- All screens render correctly on iOS 14+ and Android 8+.

□ Client Requirements for This Milestone

- Google OAuth Client ID and Client Secret — required before kickoff Monday of Week 1.
- Confirm brand colours, logo file (SVG or high-res PNG), and font preferences for Figma designs.
- App name confirmation: 'GatherGo' — needed for Splash and onboarding copy.
- AWS account access (IAM credentials) — for RDS, S3, SES, and hosting setup.
- Feedback on Figma designs within 2 business days of delivery to avoid Week 2 delays.

M2 — Home Screen & Trips

Week 2

Personalised home dashboard with quick access shortcuts, and the complete Trips module with all tabbed sections.

Milestone Goals

- Home Dashboard: personalised greeting, upcoming trips, upcoming events, ongoing section, AI shortcut icon.
- Trips list: Upcoming Trips, Past Trips, Create New Trip.
- Create New Trip modal with all fields and Google Places location search.
- Trip Detail screen: tabbed sections — Activities, Docs, Members, Photos, Expenses, Polls, Notes.
- Expense tracking inside Trip: equal/amount/% splits, balances, settle up, debt simplification.

Home Screen Specifications

Home Dashboard Sections

- Top bar: GatherGo logo left, notification bell (badge) right, profile avatar right
- Greeting card: 'Hi [Name]!' with current date and personalised subtext
- AI shortcut icon/button — taps directly into Sweet AI chat screen
- Upcoming Trips: horizontal scroll cards — trip name, destination, dates, days-to-go badge, member avatar stack
- Upcoming Events: horizontal scroll cards — event name, location, dates
- Ongoing: full-width highlighted card for any currently active trip or event
- Bottom navigation: Home | Trips | Events | Friends | Gallery

Trips Module Specifications

Trips List Screen

- Upcoming tab: full-width cards with cover photo, trip name, location, date range, days-to-go, member avatars.
- Past tab: thumbnail list — name, location, date range, member avatars.
- Create New Trip FAB (floating action button) — opens Create Trip modal.

Create New Trip Modal

- Trip Name (required, min 3 chars), Start Date, End Date, Location (Google Places autocomplete).
- Upload Docs: PDF, JPG, PNG — max 10MB per file, 5 files max, stored on AWS S3.
- Add Reminders toggle: push via FCM for trip start, 1 day before, 1 week before.

- Invite Group Members: opens Invite flow (email deep link via AWS SES, 7-day validity).
- Create Trip: creates record in AWS RDS, adds to dashboard, sends invitations.

Trip Detail Screen

- Header card: Trip Name, date range, location, Days to Go counter (large teal number).
- Stats bar: Member count | Docs | Photos | Expenses — each tappable, navigates to that tab.

Trip Detail Tabs

- Activities — Add Activity (title, date, time, location, notes). Upcoming and Completed sections with chevron toggle.
- Docs — Upload PDF/image docs (AWS S3). Download by any member. Delete by uploader or admin.
- Members — View all members with role badge (Admin/Member). Invite new (email deep link). Admin can remove members.
- Photos — 3-column grid, upload photos/videos (AWS S3), full-screen viewer with swipe, captions.
- Expenses — Add expense, split (Equal / By Amount / By %), Balances tab, Settle Up. All in INR.
- Polls — Create poll with question and options; members vote; live bar chart results.
- Notes — Shared free-form notes; any member can edit; timestamped with editor name.

Expense Tracking (inside Trips)

- Equally: fractional remainder assigned to payer. $\text{Rs. } 100 / 3 = \text{Rs. } 33.33$ each, Re. 0.01 to payer.
- By Amount: per-member INR input; submit blocked until amounts equal total exactly.
- By Percentage: per-member % input; submit blocked until percentages sum to 100%.
- Debt simplification: greedy min-cash-flow algorithm server-side on every expense mutation.
- Balances tab: Total, You Paid (green), Your Share (orange), outstanding cards per person.
- Settle Up: marks balance settled, removes outstanding card, logs timestamp and settler name.

Backend API Endpoints — M2

Method & Route	Description	Key Fields
POST /trips	Create new trip	body: {name, startDate, endDate, location, reminders} → {trip}
GET /trips	Get all trips for current user	query: {status: upcoming past} → {trips: []}
GET /trips/:id	Get single trip detail	header: auth → {trip, members, stats}
PUT /trips/:id	Edit trip (admin only)	body: {name?, startDate?, endDate?, location?} → {trip}
DELETE /trips/:id	Delete trip (admin only)	header: auth → {success: true}
POST /trips/:id/invite	Invite members via email	body: {emails: [], existingUserIds: []} → {invitesSent}
POST /trips/:id/activities	Add activity	body: {title, date, time, location, notes} → {activity}

PUT <code>/trips/:id/activities/:actId</code>	Edit activity	body: partial → {activity}
DELETE <code>/trips/:id/activities/:actId</code>	Delete activity	header: auth → {success: true}
POST <code>/trips/:id/expenses</code>	Add expense	body: {description, amount, category, paidBy, splitType, splitAmong} → {expense, balances}
GET <code>/trips/:id/expenses</code>	List trip expenses	query: {category?, page, limit} → {expenses: [], total}
PUT <code>/trips/:id/expenses/:eid</code>	Edit expense	body: partial → {expense, balances}
DELETE <code>/trips/:id/expenses/:eid</code>	Delete expense	header: auth → {success, balances}
GET <code>/trips/:id/balances</code>	Get balance summary	header: auth → {total, youPaid, yourShare, outstanding: []}
POST <code>/trips/:id/settlements</code>	Mark settlement	body: {withUserId, amount} → {settlement, updatedBalances}
POST <code>/trips/:id/polls</code>	Create poll	body: {question, options: []} → {poll}
POST <code>/trips/:id/polls/:pollId/vote</code>	Vote on poll	body: {optionId} → {poll}
GET <code>/trips/:id/notes</code>	Get shared notes	header: auth → {notes, lastEditedBy}
PUT <code>/trips/:id/notes</code>	Update shared notes	body: {content} → {notes}
POST <code>/trips/:id/docs</code>	Upload document	multipart/form-data → {doc}
GET <code>/trips/:id/docs</code>	List trip docs	header: auth → {docs: []}

Acceptance Criteria — M2

- Home Dashboard displays personalised greeting, upcoming trips, upcoming events, and ongoing section.
- AI icon on Home navigates directly to Sweet chat.
- User can create a trip; it appears on Home and Trips screens.
- Google Places returns suggestions; selecting populates coordinates in the record.
- All Trip Detail tabs are accessible and functional.
- Expense splits calculate correctly with inline validators.
- Balances update correctly after every add/edit/delete.
- Invite email sent via AWS SES within 60 seconds with working deep link.

□ Client Requirements for This Milestone

- Google Places API key with billing account enabled — required before Week 2 Monday.
- AWS SES domain/email verified for sending transactional emails (invite, reminders).
- Confirm default trip cover image placeholder (can be a solid colour or provided image).
- Approve Figma designs from M1 before development begins this week.

M3 — Events

Week 3

Complete Events module — create and manage events with shared docs, members, photos, expenses, polls, and notes. No Activities tab.

Milestone Goals

- Events list screen: Upcoming Events, Past Events, Create New Event.
- Create New Event modal with name, dates, location, docs, reminders, and member invite.
- Event Detail screen: Docs, Members, Photos, Expenses, Polls, Notes tabs.
- Events deliberately have NO Activities tab — this distinguishes them from Trips.
- Expense module inside Events is identical to the Trips expense module.
- Invite via existing user search or email deep link via AWS SES (7-day validity).

Events Module Specifications

Events List Screen

- Upcoming Events tab: cards — event name, location, date range, member avatars.
- Past Events tab: thumbnail list.
- Create New Event FAB — opens Create Event modal.

Create New Event Modal

- Event Name (required), Start Date, End Date, Location (Google Places autocomplete).
- Upload Docs: PDF, JPG, PNG — max 10MB per file, 5 files max, stored on AWS S3.
- Add Reminders toggle: push via FCM for event start and 1 day before.
- Invite Members: search existing GatherGo users or send email deep link via AWS SES.
- Create Event: saves to AWS RDS, adds to Home Dashboard, sends invitations.

Event Detail Screen

- Header card: Event Name, date range, location, Days to Go counter.
- Stats bar: Member count | Docs | Photos | Expenses — tappable.

Event Detail Tabs (no Activities tab)

- Docs — Upload/download PDF and image documents. Delete by uploader or admin.
- Members — View all event members, invite new. Admin can remove members.
- Photos — 3-column thumbnail grid, upload photos/videos, full-screen viewer with captions.
- Expenses — Full expense module: add, split (Equal / By Amount / By %), balances, settle up.
- Polls — Create poll, members vote, live bar chart results.
- Notes — Shared editable notes pad with timestamped edits.

Backend API Endpoints — M3

Method & Route	Description	Key Fields
POST /events	Create new event	body: {name, startDate, endDate, location, reminders} → {event}
GET /events	Get all events for user	query: {status: upcoming past} → {events: []}
GET /events/:id	Get single event detail	header: auth → {event, members, stats}
PUT /events/:id	Edit event (admin only)	body: partial event → {event}
DELETE /events/:id	Delete event (admin only)	header: auth → {success: true}
POST /events/:id/invite	Invite members	body: {emails: [], existingUserIds: []} → {invitesSent}
GET /events/invite/:token	Validate invite token	param: token → {event, inviterName, expiresAt}
POST /events/invite/:token/accept	Accept event invite	header: auth → {eventId, role}
POST /events/:id/expenses	Add expense to event	body: expense payload → {expense, balances}
GET /events/:id/expenses	List event expenses	query: {page, limit} → {expenses: [], total}
PUT /events/:id/expenses/:eid	Edit event expense	body: partial → {expense, balances}
DELETE /events/:id/expenses/:eid	Delete event expense	header: auth → {success, balances}
GET /events/:id/balances	Get event balances	header: auth → {total, youPaid, yourShare, outstanding: []}
POST /events/:id/settlements	Mark settlement	body: {withUserId, amount} → {settlement}
POST /events/:id/polls	Create poll	body: {question, options: []} → {poll}
POST /events/:id/polls/:pollId/vote	Vote on poll	body: {optionId} → {poll}
GET /events/:id/notes	Get shared notes	header: auth → {notes, lastEditedBy}
PUT /events/:id/notes	Update shared notes	body: {content} → {notes}
POST /events/:id/docs	Upload document	multipart/form-data → {doc}
GET /events/:id/docs	List event docs	header: auth → {docs: []}

Acceptance Criteria — M3

- User can create an event; it appears on Home Dashboard and Events list.
- Event Detail shows tabs: Docs, Members, Photos, Expenses, Polls, Notes. No Activities tab.
- Invite email sent via AWS SES within 60 seconds; deep link is valid for 7 days.
- Expense module inside Events functions identically to the Trips expense module.
- Polls allow creation, voting, and display results as a bar chart.
- Notes are shared and editable by all members with timestamped edits.

□ Client Requirements for This Milestone

- Confirm there are no additional tabs required for Events beyond the six listed.
- Provide any event-specific copy (e.g., placeholder text, empty state messages) if different from Trips.
- AWS SES sending limits — if account is in SES sandbox, request production access before this milestone.

M4 — Friends & Invite System

Week 4

Social connections: friend requests, deep link invites, and user profile with past trips and events gallery.

Milestone Goals

- Friends screen: Connected Friends list, Invite Friends flow.
- Friend request system: search by name or email, send/accept/decline, remove friend.
- Invite Friends: share referral link via device share sheet or send email via AWS SES.
- Deep link invite system for friend connections — 7-day valid token.
- Profile section: photo, name, short bio, gallery of past trips and events.

Friends Module Specifications

Friends Screen

- Connected Friends list: profile photo, name, mutual trip/event count.
- Search bar at top: find GatherGo users by name or email.
- Friend request states: Pending Sent | Pending Received | Connected.
- Accept / Decline buttons for incoming requests (push notification on receipt).
- Remove Friend option via long press or three-dot menu (confirmation dialog).

Invite Friends

- Share invite link via device share sheet (WhatsApp, SMS, copy link, etc.).
- Enter one or more email addresses to send invite via AWS SES.
- Invite link expires after 7 days; expired links show a friendly error screen.
- Users who sign up via invite link are automatically connected as friends.

Profile Section

- Accessible from profile avatar in top bar or bottom navigation.
- Circular profile photo (teal border), full name (bold), @username handle.
- Short bio / tagline, country with pin icon.
- Gallery of Past Trips: 2-column mosaic grid — trip cover photo with name overlay.
- Gallery of Past Events: 2-column mosaic grid — event cover photo with name overlay.

- Tap card: navigates to that trip or event's gallery.
- Edit Profile: update name, bio, country, profile photo (uploaded to AWS S3).

Backend API Endpoints — M4

Method & Route	Description	Key Fields
GET /friends	Get connected friends list	header: auth → {friends: [{userId, name, avatarUrl, mutualCount}]}
POST /friends/request	Send friend request	body: {toUserId} → {request}
PUT /friends/request/:reqId	Accept or decline request	body: {action: accept decline} → {friend null}
DELETE /friends/:userId	Remove friend	header: auth → {success: true}
GET /friends/requests	Get pending requests	header: auth → {incoming: [], outgoing: []}
POST /friends/invite	Send friend invite via SES	body: {emails: []} → {invitesSent: n}
GET /friends/invite/:token	Validate invite token	param: token → {inviterName, expiresAt}
POST /friends/invite/:token/accept	Accept friend invite	header: auth → {friend}
GET /users/search	Search users by name/email	query: {q} → {users: []}
GET /users/:id/profile	Get user public profile	header: auth → {name, bio, avatarUrl, country}
PUT /users/profile	Update own profile	body: {fullName?, bio?, country?} → {profile}
GET /users/:id/gallery	Get user gallery (trips + events)	header: auth → {trips: [], events: []}

Acceptance Criteria — M4

- User can search for another user and send a friend request.
- Recipient receives push notification and can accept or decline.
- Accepted request appears in Connected Friends for both users.
- Invite email sent via AWS SES with a working 7-day deep link.
- Profile shows name, bio, photo, country, and gallery of past trips and events.
- Gallery cards navigate to the correct trip or event gallery screens.

□ Client Requirements for This Milestone

- Confirm referral/invite link domain (e.g., gathergo.app/invite) — client must own and provide the domain.
- Provide any branding assets for the invite email template (logo, colour, copy tone).
- Confirm whether mutual friend count or other social metrics should display on friend cards.

M5 — Gallery & AI Chatbot (Swee)

Week 5

Media gallery for trips and events, personal profile gallery, and the Swee AI travel assistant via OpenAI GPT-4o.

Milestone Goals — Gallery

- Per-trip and per-event gallery: 3-column grid, multi-select upload, full-screen viewer.
- Video support: thumbnails with play icon overlay, inline full-screen playback.
- AWS S3 upload pipeline with per-file progress bars.
- Offline upload queue: files queue when offline, auto-sync on reconnect.
- Delete: uploader or trip/event admin only. Download: any member.
- Personal profile gallery: all user media across trips and events, organised by mosaic grid.

Gallery Screen Specifications

Per-Trip / Per-Event Gallery

- 3-column thumbnail grid, newest first. Upload (+) button top-right.
- Video thumbnails: first frame + circular play icon. Full-screen: inline video player.
- Full-screen viewer: swipeable, uploader name + date below, caption overlay on tap.
- Delete icon: uploader and admin only; confirmation dialog required before deletion.
- Download icon: all members; saves to device photo library.
- Multi-select upload up to 20 files per session; per-file and overall progress bars.
- Queued badge indicator when uploads are pending due to no connectivity.

Personal Profile Gallery

- Gallery of Trips section: count badge, 2-column mosaic, cover photo + trip name overlay.
- Gallery of Events section: count badge, 2-column mosaic, cover photo + event name overlay.
- Tap card: navigates to that trip or event's full gallery screen.

Milestone Goals — Swee AI Chatbot

- Swee chat screen: full chat UI, streaming word-by-word responses, timestamps.
- System prompt injection: trip/event destination, dates, duration, group size per request.
- Use cases: itinerary planning, day plans, restaurant recommendations, packing lists, travel Q&A.
- Attach trip/event context via + button.
- Clear conversation and report issue via overflow menu.
- Token-safe history: trim to last 20 messages if conversation exceeds 100k tokens.

Swee Chat Screen Specifications

- Accessible from AI icon on Home Dashboard and Chat tab.

- Swee avatar: teal circle with sparkle icon. Status label: 'Always active • AI Assistant'.
- Opening message: 'Hi! I'm Swee, your travel assistant. How can I help you plan your next adventure?'
- User bubble: right, teal. Swee bubble: left, light grey. Timestamps below each bubble.
- Typing indicator: animated three-dot ellipsis while waiting for first token.
- Streaming: Swee's reply appears word-by-word as OpenAI streams tokens.
- + button: attaches current trip/event context to the conversation.
- Attachment icon: send photo or doc for Swee to reference (OpenAI vision).
- Overflow menu: Clear conversation | Report issue | About Swee.

Backend API Endpoints — M5

Method & Route	Description	Key Fields
POST /ai/chat	Send message to Swee	body: {message, tripContext?, conversationHistory} → {reply, usage}
POST /ai/chat/stream	Streaming Swee response (SSE)	SSE: chunks {delta: string} → {done: true}
DELETE /ai/chat/:userId	Clear conversation history	header: auth → {success: true}
POST /ai/report	Report a Swee response	body: {messageId, reason} → {success: true}
POST /gallery/upload-url	Get pre-signed S3 upload URL	body: {fileName, mimeType, tripId eventId} → {uploadUrl, mediaId}
POST /trips/:id/gallery	Save uploaded trip media metadata	body: {mediaId, caption?} → {media}
GET /trips/:id/gallery	List trip gallery media	query: {page, limit} → {media: [], total}
DELETE /trips/:id/gallery/:mid	Delete trip media	header: auth → {success: true}
POST /events/:id/gallery	Save uploaded event media metadata	body: {mediaId, caption?} → {media}
GET /events/:id/gallery	List event gallery media	query: {page, limit} → {media: [], total}
DELETE /events/:id/gallery/:mid	Delete event media	header: auth → {success: true}

Acceptance Criteria — M5

- User selects up to 20 files; per-file progress bar visible during upload.
- Videos show play icon overlay; play inline in full-screen viewer.
- Delete icon visible only to uploader and admin; confirmation required.
- Any member can download media to their device photo library.
- Offline uploads queue with indicator badge; sync automatically on reconnect.
- Itinerary, day planning, packing list, and visa queries return appropriate responses.
- Clear conversation resets chat to welcome message.

□ Client Requirements for This Milestone

- OpenAI API key (GPT-4o access) — required before Week 5 Monday. Token costs billed to client's OpenAI account.
- Set a monthly spend cap on the OpenAI account to avoid unexpected charges.
- Confirm maximum file size cap (default: 50MB per file, 500MB per trip/event gallery).
- Confirm supported file formats (default: JPG, PNG, HEIC, MP4, MOV — HEIC auto-converted to JPG).
- AWS S3 bucket name and region preference — required before media upload implementation.

M6 — Revisions, Changes & Deployment

Week 6

Client-requested revisions, full QA & polish, AWS backend deployment, and app store submission.

Milestone Goals

- Implement all client-requested revisions and changes accumulated from Milestones 1–5.
- End-to-end QA across all five modules on iOS (iPhone 14 Pro) and Android (Pixel 7) real devices.
- All P1 (crash/blocking) bugs fixed before final demo call.
- P2 bugs fixed or documented with workarounds in handover notes.
- Backend deployed to AWS (EC2 / Elastic Beanstalk) under HTTPS with SSL via AWS ACM.
- AWS RDS production instance configured with automated backups, multi-AZ, and indexes.
- Production APK and IPA builds prepared for store submission.
- Handover documentation: README, API reference, deployment guide, environment setup guide.

Revisions & Changes

All revision feedback from Milestones 1–5 is addressed this week. This includes UI/UX tweaks, copy changes, flow adjustments, and minor functional changes within agreed scope. New features outside scope will be quoted separately before work begins.

QA & Polish

Polish Pass — All Screens

- Loading skeleton screens for all list views (trips, events, expenses, gallery, friends)
- Empty state illustrations for all empty lists with prompt copy
- Error states: network error banner with retry button on all screens
- Pull-to-refresh on all list and dashboard screens
- Haptic feedback on primary action buttons (iOS Taptic Engine, Android Vibration API)

- Smooth transitions: Hero animation on card tap, modal slide-up, tab switch fade
- Dark mode readiness: teal palette functional on both light and dark system themes
- Accessibility: minimum 44x44pt touch targets, VoiceOver and TalkBack labels on all interactive elements

App Store Submission & Testing Phases

Both stores require mandatory testing phases before a production release. These run independently from the development sprint and are outside the team's control.

Store	Testing Phase	Notes
Google Play Store	Internal Testing: ~14 days mandatory before production release. Runs independently from development sprint.	Client must have Google Play developer account (US\$25 one-time). APK submitted to Internal Track first; after 14 days, promote to production.
Apple App Store	TestFlight: 1–3 days Apple review. App Store review: 1–7 days. Timeline is Apple's discretion.	Client must have Apple Developer account (US\$99/year). IPA submitted to TestFlight first, then App Store.

Important: The 14-day Google Play internal testing phase is a mandatory Google policy and runs separately from the 6-week build. Production release on Google Play occurs after this window completes.

Backend API Endpoints — M6

Method & Route	Description	Key Fields
GET /notifications	Get user notifications	query: {page, limit, unreadOnly} → {notifications: [], unreadCount}
PUT /notifications/read-all	Mark all notifications read	header: auth → {success: true}
PUT /notifications/:nid/read	Mark single notification read	header: auth → {success: true}
GET /health	Backend health check	→ {status: 'ok', uptime, version, dbStatus}

Acceptance Criteria — M6

- All client revision requests from M1–M5 are implemented and signed off.
- Full E2E journey (signup → trip → expense → gallery → Swee) completes without crash on iOS and Android.
- App cold start to Home Dashboard under 3 seconds on mid-range Android device.
- All API endpoints respond in under 500ms for standard datasets.
- No P1 bugs in the final build.
- APK submitted to Google Play Internal Testing; 14-day testing phase documented.
- IPA submitted to Apple TestFlight and ready for App Store review.
- Backend live on AWS under HTTPS with valid SSL certificate.
- All environment variables documented in handover README.

❏ Client Requirements for This Milestone

- Apple Developer account (US\$99/year) — required before Week 6 Monday.
- Google Play developer account (US\$25 one-time) — required before Week 6 Monday.
- App signing certificates and provisioning profiles (Apple) — Zyntrix will guide setup.
- App store listing assets: app icon (1024x1024 PNG), screenshots for both stores, short description, full description, keywords.
- All revision requests from M1–M5 must be submitted in writing by end of Week 5 Friday to be included in M6.
- AWS production credentials and preferred region for final deployment.

M7 — Admin Web Panel

Week 7

Next.js web application giving the client's team full operational visibility — business insights, user management, trip/event oversight, health monitoring, storage analytics, and feedback.

Milestone Goals

- Next.js admin web application with secure admin authentication (separate from mobile auth).
- Business Insights dashboard with growth graphs for key metrics.
- Users management: list, search, view profile, change user status (active/suspended).
- Trips & Events management: view all, filter by status, view detail.
- Error / Health monitoring: failure logs and crash signals dashboard.
- Storage & Capacity analytics: consumption metrics and per-user averages.
- Feedback & Support section: view and manage user-submitted feedback.
- Admin panel deployed to AWS (separate from mobile backend) under a secure subdomain.

Admin Panel — Section Specifications

Authentication (Admin Only)

- Separate admin login page — email + password. No Google OAuth for admin.
- Admin accounts are seeded manually — no public signup.
- JWT session with shorter expiry (4 hours) for security.
- Role-based access: Super Admin (full access) and Viewer (read-only).

Business Insights Dashboard

Business Insights — Metrics & Visualisations

- Registered Users — total count, displayed as a KPI card with trend arrow

- Active Users — users active in the last 7 and 30 days, KPI card
- Trips — total trips created, KPI card
- Events — total events created, KPI card
- Growth Graphs — line charts showing Registered Users, Active Users, Trips, and Events over time (7 days / 30 days / 90 days / custom range toggle)
- All charts rendered with a charting library (e.g., Recharts or Chart.js)

Users Management

Users — Features

- List — paginated table: avatar, name, email, phone, signup date, status badge (Active / Suspended)
- Search — search by name or email with debounced real-time filtering
- User Status — toggle between Active and Suspended. Suspended users cannot log in.
- View User Detail — profile info, trip/event count, total media uploaded, account created date
- Export Users — download CSV of user list (filtered or full)

Trips / Events Management

Trips & Events — Features

- Total count KPI cards: Total | Active | Upcoming | Completed
- Trips table: name, creator, member count, start date, end date, status badge
- Events table: name, creator, member count, start date, end date, status badge
- Filter by status: Active | Upcoming | Completed
- View Trip/Event Detail: members list, expense total, photo count, doc count
- Admin can archive or delete a trip/event if required

Error / Health Monitoring

Error & Health — Features

- Failures — table of API errors logged server-side: endpoint, error code, timestamp, frequency
- Crash Signals — client-side crash reports (if crash reporting SDK integrated): device, OS, version, stack trace summary
- Health status card: backend uptime, DB connection status, S3 connectivity, last health check timestamp
- Alert badge on sidebar when error count exceeds configurable threshold

Storage & Capacity

Storage & Capacity — Metrics

- Storage Consumed — total AWS S3 storage used across all trips and events (in GB)
- Storage Available — configured S3 cap minus consumed (or 'Unlimited' if no cap set)
- Images Uploaded — total image file count across all users
- Docs Uploaded — total document file count across all users
- Avg Storage Consumption / User — total consumed divided by registered users, shown in MB
- Top consumers table — top 10 users by storage used

Feedback & Support

- List of user-submitted feedback and Swee AI issue reports.
- Each entry: user name, type (Feedback / Swee Report), message, submitted date, status (Open / Resolved).
- Admin can mark items as Resolved.
- Filter by type and status.

Backend API Endpoints — M7 (Admin)

Method & Route	Description	Key Fields
POST /admin/auth/login	Admin login	body: {email, password} → {adminId, accessToken, role}
POST /admin/auth/logout	Admin logout	header: Bearer → {success: true}
GET /admin/insights	Business insights KPIs	query: {range: 7d 30d 90d} → {registeredUsers, activeUsers, trips, events, growth: []}
GET /admin/users	List all users (paginated)	query: {page, limit, search, status} → {users: [], total}
GET /admin/users/:id	Get user detail	header: admin auth → {profile, stats}
PUT /admin/users/:id/status	Update user status	body: {status: active suspended} → {user}
GET /admin/users/export	Export users as CSV	query: {search?, status?} → CSV file stream
GET /admin/trips	List all trips	query: {status, page, limit} → {trips: [], total}
GET /admin/events	List all events	query: {status, page, limit} → {events: [], total}
DELETE /admin/trips/:id	Admin delete trip	header: admin auth → {success: true}
DELETE /admin/events/:id	Admin delete event	header: admin auth → {success: true}
GET /admin/health	System health status	→ {uptime, dbStatus, s3Status, lastChecked, errorCount}
GET /admin/errors	Error/failure log	query: {page, limit, from, to} → {errors: [], total}
GET /admin/storage	Storage analytics	→ {consumed, available, imagesCount, docsCount, avgPerUser, topConsumers: []}
GET /admin/feedback	List feedback & reports	query: {type, status, page} → {items: [], total}
PUT /admin/feedback/:id/resolve	Mark feedback resolved	header: admin auth → {success: true}

Acceptance Criteria — M7

- Admin login screen works; invalid credentials return error. Admin JWT expires in 4 hours.
- Business Insights dashboard loads KPI cards and growth line charts with 7d/30d/90d toggle.
- Users table displays all users with search and status filter working correctly.
- Admin can suspend a user; suspended users receive an error on next login attempt.
- Trips and Events tables show correct totals and filter by status.
- Error log table shows recent failures with endpoint and timestamp.
- Storage metrics display correct consumed, available, image count, doc count, and avg per user.
- Feedback section lists all user feedback; admin can mark as resolved.
- Admin panel deployed to AWS on a secure HTTPS subdomain (e.g., admin.gathergo.app).
- Admin panel is not accessible without valid admin credentials.

□ Client Requirements for This Milestone

- Confirm admin subdomain (e.g., admin.gathergo.app) — client must own and provide the domain with DNS access.
- Provide list of initial admin users (name, email) to be seeded before handover.
- Confirm admin roles needed: Super Admin and Viewer are default — notify if additional roles required.
- AWS CloudWatch or preferred logging service setup — required for error/failure log integration.
- Confirm storage cap setting for S3 (if any) to correctly calculate 'Storage Available' metric.
- Confirm if a crash reporting SDK (e.g., Sentry, Firebase Crashlytics) should be integrated in the mobile app to feed crash signals into the admin panel.

Deliverables & Handover

- Full Source Code: React Native mobile app, Node.js backend, and Next.js admin panel pushed to a private GitHub repository and access transferred to client.
- AWS RDS: PostgreSQL production instance under client's AWS account — tables, indexes, and automated backups configured.
- AWS S3: Media bucket with CORS rules, pre-signed URL config, and CloudFront CDN delivery set up under client's account.
- Android APK: Release-signed APK submitted to Google Play Internal Testing track, ready for production promotion after 14-day window.
- iOS IPA: Xcode-archived IPA on TestFlight, ready for App Store review and submission.
- Production Backend: Node.js API live on AWS under HTTPS with SSL. Environment variables configured and documented.
- Admin Panel: Next.js admin web app deployed on AWS under a secure HTTPS subdomain.
- API Reference: All REST and admin endpoints documented with request/response schemas and example calls.
- Deployment Guide: Step-by-step AWS deployment guide for backend, admin panel, RDS, and S3.

- README: Developer README covering local setup, dependencies, and running tests.
- 30-Day Free Support: Post-launch bug fixes and production issues at no extra cost. Admin accounts seeded.

Terms & Conditions

Scope

This proposal covers the seven Version 1 milestones only. Future features will be scoped separately.

Revisions

Two revision rounds per milestone are included. Additional rounds are billed at the agreed hourly rate.

Ownership

Full source code and IP ownership transfers to the client after all invoices are paid in full.

Change Requests

Any new feature or significant scope change during the engagement is assessed by the Tech Manager and quoted separately before work begins.

Delays

If a client-side dependency (API keys, accounts, approvals, feedback) is not provided on time, the project timeline shifts accordingly with no penalty to Zyntrix.

Post-Launch Support

The 30-day free support window covers bug fixes and production issues only. New feature requests during this window are billed at the agreed hourly rate.

Confidentiality

Zyntrix treats all client materials, business plans, and product specifications as strictly confidential and will not disclose them to third parties.